Package 'compfit'

July 7, 2026

Title Fit, Analyse and Compare Compartmental ODE Models (R + Julia)

Version 0.1.0

Description Build compartmental (SIR-type) ODE models from a spreadsheet specification, fit them by maximum likelihood (L-BFGS-B / DEoptim / hypercube) or Bayesian sampling, and analyse the results: posterior summaries, prior-vs-posterior and predictive plots, global (Sobol) and local (derivative) sensitivity, practical identifiability, and self-contained code export (counterfactual scenario analysis is planned for a future release). ODE solves and Bayesian sampling run in Julia via the JuliaCall bridge for speed (and gradient-based NUTS sampling), but Julia is optional: a pure-R backend (deSolve, with gradient-free MCMC for the Bayesian case) fits without it, and a Stan backend (rstan) offers gradient-based NUTS with no Julia.

License GPL (>= 3)

Encoding UTF-8

SystemRequirements Julia (>= 1.6); Julia packages OrdinaryDiffEq, Turing, Distributions (installed on first setup_julia()).

Imports deSolve,
ggplot2,
grid,
readxl,
DEoptim,
pracma,
qrng,
grDevices,
stats,
utils

Suggests JuliaCall,
diffeqr,
testthat (>= 3.0.0),
sensitivity,
patchwork,
writexl,
openxlsx,
readr,
arrow,
jsonlite,
xml2,

scales,
beepr,
BayesianTools,
coda,
rstan,
knitr,
rmarkdown,
DiagrammeR,
svglite

VignetteBuilder knitr

Config/testthat/edition 3

RoxygenNote 7.3.2

Contents

`bayes_control` *Bayesian sampling settings*

Description

Bundle the settings for ‘method = "bayes"‘. These apply across all three Bayesian backends selected by [solver_control()]: Julia/Turing NUTS (‘backend = "julia"‘), Stan/rstan NUTS (‘backend = "stan"‘), and the gradient-free ‘BayesianTools‘ sampler (‘backend = "r"‘). Priors themselves are read from the model sheet’s ‘Parameters‘/‘States‘ columns (per-quantity boxes or named distributions); the settings here govern the sampler, not the priors.

Usage

```
bayes_control(
  sampler = "NUTS(0.65)",
  chains = 4,
  iter = 2000,
  warmup = 1000,
  seed = NULL,
  init_from_optim = TRUE,
  progress = TRUE,
  sigma_prior = "truncated(Normal(0, 1), 0, Inf)",
  phi_prior = "Gamma(2, 5)",
  sigma_prior_stan = "normal(0, 1)",
  phi_prior_stan = "gamma(2, 0.2)",
  adapt_delta = 0.8
)
```

Arguments

`sampler`	Julia/Turing sampler expression, e.g. ‘"NUTS(0.65)"‘ (the argument is the target acceptance rate). Used by ‘backend = "julia"‘ only; Stan uses its own NUTS and the R backend uses DEzs.
`chains`	Number of independent MCMC chains (default 4). Multiple chains enable the R-hat convergence diagnostic.
`iter`	Total iterations per chain, **including** warmup (default 2000).
`warmup`	Warmup / burn-in iterations per chain, discarded before inference (default 1000); the kept sample size per chain is ‘iter - warmup‘.
`seed`	Optional RNG seed for reproducible sampling.

`init_from_optim`
: If ‘TRUE‘ (default), run a quick MLE fit first and initialise the sampler there (a MAP-style warm start). Applies to the Julia and R backends; the Stan backend relies on Stan’s own warmup adaptation and ignores it.

`progress`
: Show sampler progress. ‘FALSE‘ fully silences it for every backend: the Turing ‘Sampling ... ETA‘ bar (via ‘Turing.setprogress¡), and Stan’s per-iteration refresh, chain messages, and auto-opened progress window. (The MLE optimiser’s progress is controlled separately, by [optim_control()]’s ‘progress‘.)

`sigma_prior`
: Prior for the Gaussian/lognormal noise SD, as a Julia expression (default a half-Normal). Used by the Julia backend.

`phi_prior`
: Prior for the negative-binomial dispersion, as a Julia expression. Used by the Julia backend.

`sigma_prior_stan`, `phi_prior_stan`
: The same two priors written as **Stan** expressions (defaults ‘"normal(0, 1)"‘ and ‘"gamma(2, 0.2)"‘). Used by the Stan backend, which cannot read the Julia-syntax versions above.

`adapt_delta`
: Stan NUTS target acceptance probability (default 0.8). Raise toward 1 (e.g. 0.95) to reduce divergent transitions. Stan backend only.

Value

A named list of Bayesian settings, passed as ‘bayes =‘ to [fitCompartmentalModel()].

See Also

[fitCompartmentalModel()], [solver_control()], [optim_control()].

Examples

```
bayes_control(chains = 4, iter = 2000, warmup = 1000)
bayes_control(progress = FALSE)              # silent sampling
bayes_control(chains = 2, iter = 1000, seed = 1)  # reproducible, lighter run
```

`build_compartmental_model` *Build a compartmental model (no fitting)*

Description

Runs everything [fitCompartmentalModel()] does except the fit dispatch: builds the model, time grid, prepared data, bounds and loss closure. Useful for tools that need the model and bounds without a prior fit (e.g. Sobol sensitivity).

Usage

```
build_compartmental_model(
  modelParams,
  dataCombined = NULL,
  solver = solver_control(),
  init = NULL,
  checkpoint_file = tempfile(fileext = ".rds")
)
```

Arguments

modelParams	Model parameter sheet (data frame).
dataCombined	Combined observation data (data frame). Optional: pass 'NULL' (the default) to build in simulation mode with no observed streams, in which case the loss closure is 'NULL' (nothing to fit against). See [simulate_model()].
solver	Solver settings, see [solver_control()].
init	Optional initial point (named numeric) for the loss closure.
checkpoint_file	Path for the best-solution checkpoint '.rds'.

Value

A list of class '"compartmentalModel"' (model, sap, time_grid, data, bounds, loss, best_state, best_start, solver). 'loss' is 'NULL' when 'dataCombined' is absent.

Examples

```
mini <- system.file("extdata", "minimal", package = "compfit")
sc   <- load_scenario(mini, combined_file = "dataCombined.csv",
                      dummy_file = "dataDummy.csv", params_file = "modelParams.csv")
# Build the machinery (no fitting). The R backend needs no Julia:
m <- build_compartmental_model(sc$modelParams, sc$dataCombined,
                               solver = solver_control(backend = "r"))
m$bounds
```

---

cfit_palette *compfit colour palette*

Description

Named list of hex colours (Okabe-Ito based) used across compfit plots: edit this to restyle data, model, band, censor and dummy elements.

Usage

```
cfit_palette
```

Format

A named list of hex colour strings.

Examples

```
cfit_palette$data
```

cfit_palette_grey *compfit grayscale palette*

Description

A grayscale counterpart to [cfit_palette], for print/monochrome output. Same named slots (‘data‘, ‘model‘, ‘band‘, ‘censor‘, ‘dummy‘); elements are kept apart by shade (and by shape/linetype in the plots). Select it with ‘plot_fit(..., palette = "grey")‘ or globally via ‘options(compfit.palette = "grey")‘.

Usage

```
cfit_palette_grey
```

Format

A named list of hex colour strings.

Examples

```
cfit_palette_grey$data
```

cfit_theme *compfit ggplot2 theme*

Description

A minimal ggplot2 theme used across compfit plots.

Usage

```
cfit_theme(base_size = 11)
```

Arguments

base_size	Base font size.

Value

A ggplot2 theme object.

Examples

```
library(ggplot2)
ggplot(mtcars, aes(wt, mpg)) + geom_point() + cfit_theme()
```

`compare_to_mle` *Compare a Bayesian fit to an MLE fit*

Description

Tabulates posterior mean and median against the MLE point estimate, with relative differences for each.

Usage

```
compare_to_mle(fit, fit_mle)
```

Arguments

`fit`	A Bayesian ‘"compartmentalFit"‘ object.
`fit_mle`	A maximum-likelihood ‘"compartmentalFit"‘ object.

Value

A data frame with ‘parameter‘, ‘bayes_mean‘, ‘bayes_median‘, ‘mle‘, and relative-difference columns.

Examples

```
## Not run:
compare_to_mle(fit_b, fit_mle)   # Bayesian vs maximum-likelihood fit

## End(Not run)
```

`compartmentalFit` *The ‘compartmentalFit‘ object*

Description

The object returned by [fitCompartmentalModel()]. Its components are public, but the accessors (see [compfit-accessors]) provide a stable, validated interface and are the recommended way to reach them.

Components

‘method‘, ‘success‘, ‘error_msg‘ Run metadata.

‘point‘ *(point methods only)* ‘list(initial_state, parms)‘ on the natural scale.

‘best_state‘ Environment with ‘$error‘ (best loss value reached) and ‘$par‘ (best normalised parameter vector).

‘fit_raw‘ Raw optimiser/sampler return (‘optim‘, ‘DEoptim‘, or a Turing chain list), or a ‘fitError‘ object on failure.

‘samples‘ *(bayes only)* list with: ‘draws‘ (one row per joint MCMC sample; fitted parameters as normalised ‘name_n‘ columns, hyperparameters on natural scale — use [posterior_draws()] to convert all to natural scale), ‘summary‘ (‘parameter, mean, sd, rhat, ess‘), ‘prior_spec‘, ‘like_specs‘, ‘model_code‘ (generated Turing ‘@model‘ source), ‘scale_cols‘, ‘stream_names‘, ‘n_chains‘, ‘iter‘.

‘loss‘ The loss closure (dropped on ‘save_fit()‘, rebuilt by [load_fit()] on load).

‘model‘ List: ‘structure‘, ‘sap‘, ‘expressions‘, ‘compartmental_function‘ (R closure), ‘julia_code‘ (character), ‘date‘, and ‘modelParams‘ (the source sheet).

‘data‘ Prepared data; all matrices are years x streams: ‘data_combined‘, ‘matrix_data_points‘, the observation/censoring masks (‘obs_mask‘; ‘cens_mask‘/‘limit_mat‘ for left-censored ‘<L‘/‘<=L‘; ‘lcens_mask‘/‘llimit_mat‘ for right-censored ‘>L‘/‘>=L‘), ‘weight_matrix‘, ‘average_matrix‘, ‘names_data_points‘, and ‘cumulative_cols‘ (stream indices differenced from cumulative to annual).

‘time_grid‘ List: ‘time‘, ‘startpoint‘, ‘endpoint‘, ‘partition‘, ‘cutoff‘.

‘bounds‘ List: ‘lower‘, ‘upper‘, ‘init_norm‘.

‘solver‘ The [solver_control()] list used.

‘meta‘ List: ‘checkpoint_file‘.

See Also

[fitCompartmentalModel()], [compfit-accessors], [save_fit()], [load_fit()], [summary.compartmentalFit()], [print.compartmentalFit()]

`compfit-accessors` *Accessors for a compartmental fit*

Description

Extract components of a ‘"compartmentalFit"‘ object.

Usage

```
get_loss(fit)

get_model(fit)

get_data(fit)

get_time_grid(fit)

get_bounds(fit)

get_solver(fit)

get_compartmental_function(fit)

get_julia_code(fit)

get_expressions(fit)

get_states_and_params(fit)

get_point(fit)

get_samples(fit)
```

Arguments

fit	A ‘"compartmentalFit"‘ object.

Value

The requested component of the fit.

Examples

```
## Not run:
# fit from fitCompartmentalModel()
get_point(fit)
get_julia_code(fit)

## End(Not run)
```

---

extract_code *Export a fit as self-contained source code*

Description

Emits portable R source that reproduces the model, loss, and/or fit from a ‘"compartmentalFit"‘ (or model) object, inlining the captured data so the script runs in a fresh session (given a Julia bridge).

Usage

```
extract_code(
  x,
  file = NULL,
  what = c("all", "model", "loss", "fit"),
  inline = c("readable", "fidelity")
)
```

Arguments

x	A ‘"compartmentalFit"‘ or ‘"compartmentalModel"‘ object.
file	Optional output path; if ‘NULL‘, returns the code as a string.
what	Which part(s) to emit: all, model, loss, or fit.
inline	How captured data objects (bounds, data matrices, masks, ‘partition‘, ‘init_norm‘, ...) are written into the script. ‘"readable"‘ (default) emits them as human-readable, re-parseable ‘deparse()‘ code (names, dims, NAs preserved; doubles to 17 significant digits, which round-trips exactly for typical values but can differ by ~1 ULP on adversarial doubles, as R’s decimal parser is not always correctly rounded). ‘"fidelity"‘ emits them as opaque ‘unserialize(as.raw(...))‘ blobs — a guaranteed byte-exact round-trip, for when the last bit matters.

Value

The generated code as a character string, invisibly (also written to ‘file‘ when supplied).

Examples

```
## Not run:
extract_code(fit, file = file.path(tempdir(), "reproduce.R"), what = "all")
# opaque-but-byte-exact data blocks:
extract_code(fit, file = file.path(tempdir(), "exact.R"), inline = "fidelity")

## End(Not run)
```

`fill_params` *Write fitted estimates back into a model sheet*

Description

Returns a copy of the model parameter sheet with fitted States/Parameters replaced by their point estimates (fixing them as '*name=value'); structural columns are left intact.

Usage

```
fill_params(fit, modelParams, digits = NULL, summary = c("median", "mean"))
```

Arguments

`fit`	A '"compartmentalFit"' object.
`modelParams`	The model parameter sheet (data frame) to fill.
`digits`	Optional rounding for written values.
`summary`	Which point estimate to write ('"median"' or '"mean"').

Value

The filled model parameter data frame.

Examples

```
## Not run:
# fit + modelParams from a fitted scenario
filled <- fill_params(fit, sc$modelParams, digits = 4)

## End(Not run)
```

`fitCompartmentalModel` *Fit a compartmental model*

Description

Build a compartmental ODE model from a spreadsheet specification and fit it by maximum likelihood or Bayesian sampling. By default ODE solves (and Julia NUTS) run in Julia; call [setup_julia()] first or rely on the lazy initialiser. A pure-R backend and a Stan backend need no Julia – see [solver_control()].

Usage

```
fitCompartmentalModel(
  modelParams,
  dataCombined,
  method = c("lbfgsb", "deoptim", "hypercube", "bayes"),
  solver = solver_control(),
  control = optim_control(),
  hypercube = hypercube_control(),
  bayes = NULL,
  init = NULL,
  checkpoint_file = "checkpoint_best_solution.rds",
  debug_env = NULL
)
```

Arguments

`modelParams`	Model-parameter sheet (data frame): compartments, priors, rate coefficients and the time grid. See ‘vignette("compfit")‘ for the grammar.
`dataCombined`	Observed data, one row per stream (‘Formula‘ + one column per year, optional ‘Label‘/‘Likelihood‘/‘Weight‘/‘Average‘). Required for fitting; to run a fully-fixed model forward with no data use [simulate_model()] instead.
`method`	Fitting method (one of): **‘"lbfgsb"‘** deterministic local MLE (L-BFGS-B). Fast; the default. **‘"deoptim"‘** stochastic global MLE (differential evolution). More robust on rugged/multi-modal objectives, slower. Seed via [optim_control()]. **‘"hypercube"‘** deterministic Sobol pre-search; a coarse global scan (see [hypercube_control()]). **‘"bayes"‘** full Bayesian posterior sampling (see [bayes_control()] and the ‘backend‘ in [solver_control()]).
`solver`	Solver/backend settings, see [solver_control()] – chooses ‘"julia"‘ / ‘"r"‘ / ‘"stan"‘ and the ODE tolerances.
`control`	Optimiser settings for the MLE methods, see [optim_control()].
`hypercube`	Pre-search settings for ‘method = "hypercube"‘, see [hypercube_control()].
`bayes`	Sampler settings, see [bayes_control()]; used only for ‘method = "bayes"‘ (defaults are supplied if ‘NULL‘).
`init`	Optional starting point as a named numeric vector on the normalised \[0,1\] scale (advanced; normally left ‘NULL‘).

checkpoint_file
: Path to an ‘.rds‘ where the running best solution is written during an MLE search, so a long fit can be recovered if interrupted.

debug_env
: Optional environment into which intermediate build objects are stashed for debugging (no effect on the result when ‘NULL‘).

Value

An object of class ‘"compartmentalFit"‘: ‘$point‘ (natural-scale estimates) for MLE, ‘$samples‘ for Bayes, plus the model, data, bounds and solver used. Inspect with [summary()], [get_point()], [posterior_report()], [plot_fit()].

Examples

```
mini <- system.file("extdata", "minimal", package = "compfit")
sc   <- load_scenario(mini, combined_file = "dataCombined.csv",
                      dummy_file = "dataDummy.csv", params_file = "modelParams.csv")
# Julia-free maximum-likelihood fit (pure-R deSolve backend) -- runs at check:
fit <- fitCompartmentalModel(sc$modelParams, sc$dataCombined, method = "lbfgsb",
                             solver = solver_control(backend = "r"),
                             checkpoint_file = tempfile(fileext = ".rds"))
get_point(fit)
## Not run:
# The Julia backend: faster, and required for NUTS Bayesian sampling.
setup_julia()
fit_b <- fitCompartmentalModel(sc$modelParams, sc$dataCombined, method = "bayes",
                               bayes = bayes_control(chains = 2, iter = 1000))

## End(Not run)
```

---

hypercube_control *Hypercube pre-search settings*

Description

Settings for the quasi-Monte Carlo (Sobol) hypercube pre-search (‘method = "hypercube"‘), which evaluates the loss on a low-discrepancy Sobol design over the normalised \[0,1\] box and keeps the best point. Deterministic and derivative-free – useful as a stand-alone coarse fit or to seed a warm start for the local optimiser.

Usage

```
hypercube_control(n = 10000)
```

Arguments

n
: Number of Sobol sample points to evaluate (default 10000). More points give finer coverage of the parameter box at linear cost in solves. n should approximately be of magnitude 10^m where m is the number of unknown parameters and initial states.

Value

A named list with the sample count, passed as ‘hypercube =‘ to [fitCompartmentalModel()].

See Also

[fitCompartmentalModel()], [optim_control()].

Examples

```
hypercube_control(n = 5000)
```

`identifiability_report` *Practical identifiability report*

Description

Diagnoses practical identifiability of a Bayesian fit: prior-to-posterior narrowing, posterior coverage of the prior range, and posterior parameter correlations, flagging weakly-identified or correlated parameters.

Usage

```
identifiability_report(
  fit,
  components = c("narrowing", "coverage", "correlation"),
  cor_threshold = 0.8,
  coverage_threshold = 0.7,
  cri = 0.95,
  plots = TRUE
)
```

Arguments

`fit`	A Bayesian ‘"compartmentalFit"‘ object.
`components`	Which diagnostics to run: narrowing, coverage, correlation.
`cor_threshold`	Absolute correlation above which a pair is flagged.
`coverage_threshold`	Posterior/prior coverage above which a parameter is flagged as poorly narrowed.
`cri`	Central credible mass used for the diagnostics (default ‘0.95‘).
`plots`	If ‘TRUE‘, include diagnostic plots.

Value

Invisibly, a list of the diagnostic tables (and plots).

Examples

```
## Not run:
identifiability_report(fit_b)   # fit_b a method = "bayes" fit

## End(Not run)
```

load_fit *Load a fit from disk*

Description

Reads a ‘"compartmentalFit"‘ saved by [save_fit()] and rebuilds the loss closure, re-registering the Julia ODE function in the running session.

Usage

```
load_fit(path)
```

Arguments

path	Path to the ‘.rds‘ file (‘.rds‘ appended if missing).

Value

A ‘"compartmentalFit"‘ object.

Examples

```
## Not run:
setup_julia()
fit <- load_fit(file.path(tempdir(), "fit.rds"))

## End(Not run)
```

load_scenario *Load a scenario folder*

Description

Reads the input workbooks (combined data, optional dummy data, model parameter sheet) from a scenario directory and assembles the inputs needed by [fitCompartmentalModel()] / [build_compartmental_model()], plus an output plot path.

Usage

```
load_scenario(
  scenario_dir,
  combined_file = "dataCombined.xlsx",
  dummy_file = "dataDummy.xlsx",
  params_file = "modelParams.xlsx",
  plots_subdir = "Plots",
  plot_file = "Plot.pdf"
)
```

Arguments

scenario_dir	Folder containing the scenario inputs.
combined_file	Combined-data workbook name.
dummy_file	Optional dummy-data workbook name.
params_file	Model parameter sheet name.
plots_subdir	Sub-folder for output plots.
plot_file	Default output plot file name.

Value

A list with the loaded ‘dataCombined‘, ‘dataDummy‘, ‘modelParams‘, and the resolved plot path.

Examples

```
# The package ships a minimal example scenario as csv:
mini <- system.file("extdata", "minimal", package = "compfit")
sc <- load_scenario(mini,
                    combined_file = "dataCombined.csv",
                    dummy_file    = "dataDummy.csv",
                    params_file   = "modelParams.csv")
str(sc, max.level = 1)
```

---

local_sensitivity *Local (derivative) sensitivity at a fit*

Description

Finite-difference sensitivity of each model output to the fitted parameters, evaluated at the fitted point. ‘type‘ selects absolute derivatives, relative elasticities, or semi-relative (parameter-scaled) sensitivities.

Usage

```
local_sensitivity(
  fit,
  type = c("relative", "absolute", "semi"),
  output = c("both", "summary", "trajectory"),
  summary_fun = mean,
  eps = 1e-04,
  data_dummy = NULL,
  progress = TRUE
)
```

Arguments

fit	A ‘"compartmentalFit"‘ object.
type	Sensitivity type: relative (elasticity), absolute, or semi.
output	What to return: both, summary, or trajectory.
summary_fun	Function summarising each output trajectory to a scalar.

eps	Finite-difference step on the normalised scale.
data_dummy	Optional dummy-data data frame.
progress	Show a progress bar.

Value

A list with ‘summary‘ and/or ‘trajectory‘ tidy data frames.

Examples

```
## Not run:
ls <- local_sensitivity(fit, type = "relative")   # fit from fitCompartmentalModel()

## End(Not run)
```

---

next_plot_path *Next numbered plot path*

Description

Given a plot path, returns the next available numbered variant (e.g. ‘Plot_1.pdf‘, ‘Plot_2.pdf‘, ...) so existing plots are never overwritten.

Usage

```
next_plot_path(path)
```

Arguments

path	A plot file path.

Value

A character path with the next free numeric suffix.

Examples

```
next_plot_path(file.path(tempdir(), "Plot.pdf"))
```

`optim_control` *Optimiser settings*

Description

Bundle optimiser settings for the maximum-likelihood methods. Two optimisers share this control: the deterministic quasi-Newton **L-BFGS-B** (‘method = "lbfgsb"‘, gradient by finite differences) and the stochastic **DEoptim** differential-evolution search (‘method = "deoptim"‘, a global method that is more robust to multi-modal / rugged objectives but slower). The ‘maxit‘/‘factr‘ options apply to L-BFGS-B; ‘itermax‘/‘NP_mult‘/‘CR‘/‘F‘/ ‘trace‘/‘seed‘ apply to DEoptim; ‘progress‘ applies to both. Settings for the other optimiser are simply ignored, so one control object works for any MLE method.

Usage

```
optim_control(
  opt_method = "L-BFGS-B",
  maxit = 1000,
  factr = 1e+07,
  itermax = 1000,
  NP_mult = 10,
  CR = 0.9,
  F = 0.8,
  trace = 50,
  seed = NULL,
  progress = TRUE
)
```

Arguments

`opt_method`	The ‘optim()‘ method used on the ‘lbfgsb‘ path. Normally ‘"L-BFGS-B"‘ (box-constrained); ‘"Brent"‘ is the 1-D alternative. The search runs over a normalised \[0,1\] box, so the method must support box bounds.
`maxit`	L-BFGS-B: maximum iterations before it stops (default 1000).
`factr`	L-BFGS-B: convergence tolerance as a multiple of machine epsilon; smaller = stricter/slower (default 1e7, i.e. ~1e-8 relative).
`itermax`	DEoptim: maximum number of generations (default 1000).
`NP_mult`	DEoptim: population size as a multiple of the number of fitted quantities (‘NP = NP_mult * d‘). The DEoptim authors recommend >= 10 (the default); lower is faster but explores less.
`CR`	DEoptim: crossover probability in \[0,1\] (default 0.9).
`F`	DEoptim: differential weighting (mutation) factor, typically in \[0,2\] (default 0.8).
`trace`	DEoptim: print the best value every ‘trace‘ generations (default 50). Ignored when ‘progress = FALSE‘.
`seed`	Optional RNG seed for the stochastic ‘deoptim‘ method, for reproducible fits. ‘lbfgsb‘/‘hypercube‘ are already deterministic and ignore it – except that a ‘|random‘ parameter start (see the modelParams grammar in ‘vignette("compfit")‘) draws its L-BFGS-B start from this seed, so setting it makes that random warm start reproducible too.

progress — Show optimisation progress. ‘TRUE‘ (default) prints the running best objective each time the loss improves and (for DEoptim) its per-generation ‘trace‘. ‘FALSE‘ silences BOTH for a quiet fit – the best solution is still tracked and recovered. (The Bayesian sampler’s progress is controlled separately, by [bayes_control()]’s ‘progress‘.)

Value

A named list of optimiser settings, passed as ‘control =‘ to [fitCompartmentalModel()].

See Also

[fitCompartmentalModel()], [bayes_control()], [hypercube_control()].

Examples

```
optim_control(maxit = 500)                       # L-BFGS-B, 500 iterations
optim_control(opt_method = "DEoptim", itermax = 200)  # DE, 200 generations
optim_control(progress = FALSE)                  # quiet fit (no console output)
optim_control(seed = 1)                          # reproducible DEoptim
```

---

plot_fit *Plot a compartmental fit*

Description

One panel per data stream: observed points, the fitted trajectory, and (optionally) uncertainty bands or spaghetti draws for Bayesian fits.

Usage

```
plot_fit(
  fit,
  bands = TRUE,
  band_type = c("predictive", "mean", "both", "spaghetti"),
  n_draws = 200,
  n_rep = 5,
  spaghetti_draws = 100,
  spaghetti_predictive = FALSE,
  base_size = 11,
  ncol = NULL,
  data_dummy = NULL,
  palette = NULL
)
```

Arguments

fit — A ‘"compartmentalFit"‘ object.

bands — Whether to draw uncertainty bands (Bayesian fits).

band_type — Band style: predictive, mean, both, or spaghetti.

n_draws — Posterior draws used for band construction.

n_rep	Replicates per draw for predictive bands.
spaghetti_draws	Number of spaghetti trajectories.
spaghetti_predictive	If ‘TRUE‘, spaghetti lines include observation noise.
base_size	Base font size.
ncol	Number of facet columns (default: automatic).
data_dummy	Optional dummy-data data frame to overlay.
palette	Colour scheme: ‘"okabe"‘ (default, colour-blind-safe) or ‘"grey"‘/‘"grayscale"‘ for monochrome output; also accepts a custom palette list (see [cfit_palette]). ‘NULL‘ honours ‘options(compfit.palette=)‘.

Value

A list with per-stream ‘plots‘ and (if patchwork is available) a combined ‘grid‘, plus the plotting ‘code‘.

Examples

```
## Not run:
res <- plot_fit(fit, band_type = "both")       # fit from fitCompartmentalModel()
res$grid
res_bw <- plot_fit(fit, palette = "grey")      # monochrome
options(compfit.palette = "grey")              # ... or switch every plot

## End(Not run)
```

---

plot_local_sensitivity

Plot local sensitivity

Description

Visualises a [local_sensitivity()] result as a tornado plot (ranked summary sensitivities) or a trajectory plot (sensitivity over time).

Usage

```
plot_local_sensitivity(
  ls,
  kind = c("tornado", "trajectory"),
  top = NULL,
  base_size = 11,
  ncol = NULL
)
```

Arguments

ls	A [local_sensitivity()] result.
kind	Plot kind: tornado or trajectory.
top	Optional number of top parameters to show.
base_size	Base font size.
ncol	Number of facet columns (default: automatic).

Value

A ggplot object (or a list of them).

Examples

```
## Not run:
plot_local_sensitivity(ls, kind = "tornado")   # ls from local_sensitivity()

## End(Not run)
```

---

plot_prior_posterior *Plot priors against posteriors*

Description

One panel per fitted parameter overlaying the prior density and the posterior histogram, with a credible interval marked.

Usage

```
plot_prior_posterior(fit, which = NULL, ncol = 4, bins = 30, cri = 0.95)
```

Arguments

fit	A Bayesian ‘"compartmentalFit"‘ object.
which	Optional subset of parameter names to plot.
ncol	Number of facet columns.
bins	Histogram bin count.
cri	Central credible mass to mark (default ‘0.95‘).

Value

A list with per-parameter ‘plots‘ and (if available) a combined ‘grid‘.

Examples

```
## Not run:
plot_prior_posterior(fit_b)   # fit_b a method = "bayes" fit

## End(Not run)
```

`plot_simulation` *Plot a simulated model*

Description

Plot the trajectories from a [simulate_model()] result: one panel per compartment and/or per evaluated formula (dummy / overlay series), each a line over the date grid. Mirrors [plot_fit()]'s return shape.

Usage

```
plot_simulation(
  sim,
  which = c("both", "states", "series"),
  palette = NULL,
  ncol = 2,
  base_size = 11
)
```

Arguments

`sim`	A ‘"compartmentalSim"‘ object from [simulate_model()].
`which`	Which panels to draw: ‘"states"‘ (compartment trajectories), ‘"series"‘ (evaluated dummy/overlay formulas), or ‘"both"‘ (default).
`palette`	Colour palette: a scheme name (‘"okabe"‘/‘"grey"‘), a custom list like [cfit_palette], or ‘NULL‘ to use the ‘compfit.palette‘ option.
`ncol`	Number of columns in the arranged grid.
`base_size`	Base font size for the theme.

Value

A list with ‘plots‘ (named list of ggplot objects) and ‘grid‘ (a patchwork arrangement, or ‘NULL‘ if patchwork is unavailable).

See Also

[simulate_model()], [plot_fit()].

Examples

```
sim_dir <- system.file("extdata", "SIR_sim", package = "compfit")
mp <- read_data_file(file.path(sim_dir, "modelParams.csv"))
dd <- read_data_file(file.path(sim_dir, "dataDummy.csv"))
sim <- simulate_model(mp, data_dummy = dd, solver = solver_control(backend = "r"))
## Not run:  plot_simulation(sim)$grid
```

posterior_draws *Posterior draws (natural scale)*

Description

Returns the posterior draws with each normalised ‘name_n‘ column mapped back to its natural scale ‘name‘ on ‘[lo, hi]‘; non-normalised columns (e.g. ‘sigma‘, ‘phi‘) pass through unchanged.

Usage

```
posterior_draws(fit)
```

Arguments

fit	A Bayesian ‘"compartmentalFit"‘ object.

Value

A data frame of natural-scale draws.

Examples

```
## Not run:
d <- posterior_draws(fit_b)   # fit_b a method = "bayes" fit
head(d)

## End(Not run)
```

posterior_intervals *Posterior credible intervals (natural scale)*

Description

Posterior credible intervals (natural scale)

Usage

```
posterior_intervals(fit, p = 0.95)
```

Arguments

fit	A Bayesian ‘"compartmentalFit"‘ object.
p	Central credible mass (default ‘0.95‘).

Value

A data frame with lower/upper interval bounds per parameter.

Examples

```
## Not run:
posterior_intervals(fit_b, p = 0.9)   # fit_b a method = "bayes" fit

## End(Not run)
```

`posterior_means` *Posterior means (natural scale)*

Description

Posterior means (natural scale)

Usage

```
posterior_means(fit)
```

Arguments

`fit`	A Bayesian ‘"compartmentalFit"‘ object.

Value

A named numeric vector of posterior means.

Examples

```
## Not run:
posterior_means(fit_b)   # fit_b a method = "bayes" fit

## End(Not run)
```

`posterior_report` *Report on a Bayesian posterior*

Description

Prints a posterior summary and convergence diagnostics (R-hat / ESS) with threshold flags, and returns the summary invisibly.

Usage

```
posterior_report(fit, p = 0.95, rhat_thresh = 1.01, ess_thresh = 400)
```

Arguments

`fit`	A Bayesian ‘"compartmentalFit"‘ object.
`p`	Central credible mass (default ‘0.95‘).
`rhat_thresh`	R-hat threshold above which a parameter is flagged.
`ess_thresh`	Effective-sample-size threshold below which a parameter is flagged.

Value

The posterior summary data frame, invisibly.

Examples

```
## Not run:
posterior_report(fit_b)   # fit_b a method = "bayes" fit

## End(Not run)
```

`posterior_summary` *Posterior summary (natural scale)*

Description

Per-parameter posterior summary (mean, sd, rhat, ess) with means/sds mapped to the natural scale.

Usage

```
posterior_summary(fit)
```

Arguments

`fit`	A Bayesian ‘"compartmentalFit"‘ object.

Value

A data frame, one row per parameter.

Examples

```
## Not run:
posterior_summary(fit_b)   # fit_b a method = "bayes" fit

## End(Not run)
```

`print.compartmentalFit`
Print a compartmental fit

Description

Print a compartmental fit

Usage

```
## S3 method for class 'compartmentalFit'
print(x, ...)
```

Arguments

x	A ‘"compartmentalFit"‘ object.
...	Unused.

Value

‘x‘, invisibly.

Examples

```
## Not run:
fit   # fit from fitCompartmentalModel(); auto-prints via this method

## End(Not run)
```

---

read_data_file *Read a data file (csv or xlsx)*

Description

Reads a ‘.csv‘ or ‘.xlsx‘ data file, optionally forcing every column to be read as text so numbers stored as text round-trip unchanged.

Usage

```
read_data_file(path, text_cols = FALSE, ...)
```

Arguments

path	Path to a ‘.csv‘ or ‘.xlsx‘ file.
text_cols	If ‘TRUE‘, read all columns as character.
...	Passed to the underlying reader.

Value

A data frame.

Examples

```
f <- system.file("extdata", "minimal", "modelParams.csv", package = "compfit")
read_data_file(f, text_cols = TRUE)
```

registerJuliaODEFunction *Register the ODE function in Julia*

Description

Low-level solver bridge: defines the generated pure-Julia ODE function (‘compartmental_function_jl‘) in the running Julia session so [solveWithJulia()] can call it. Exported because the self-contained scripts emitted by [extract_code()] call it after ‘library(compfit)‘.

Usage

```
registerJuliaODEFunction(julia_code)
```

Arguments

julia_code	Character; the Julia ODE source (e.g. ‘get_julia_code(fit)‘).

Value

‘TRUE‘, invisibly (called for its side effect in the Julia session).

reload_fit *Reload a fit (optionally re-registering Julia)*

Description

Entry point used by the counterfactual pipeline and interactive reloads. With ‘register = TRUE‘ returns a fully re-executable fit (re-registers the Julia ODE and rebuilds the loss via [load_fit()]); with ‘register = FALSE‘ does a results-only read (object + draws, no Julia session).

Usage

```
reload_fit(path, register = TRUE)
```

Arguments

path	Path to the ‘.rds‘ file (‘.rds‘ appended if missing).
register	If ‘TRUE‘, re-register Julia and rebuild the loss.

Value

A ‘"compartmentalFit"‘ object.

Examples

```
## Not run:
fit  <- reload_fit(file.path(tempdir(), "fit.rds"))                 # with Julia
draws <- reload_fit(file.path(tempdir(), "fit.rds"), register = FALSE) # results only

## End(Not run)
```

save_fit *Save a fit to disk*

Description

Serialises a ‘"compartmentalFit"‘ to an ‘.rds‘ file (the non-portable loss closure is dropped; [load_fit()] rebuilds it).

Usage

```
save_fit(fit, path)
```

Arguments

fit	A ‘"compartmentalFit"‘ object.
path	Output path (‘.rds‘ appended if missing).

Value

The output path, invisibly.

Examples

```
## Not run:
save_fit(fit, file.path(tempdir(), "fit.rds"))   # fit from fitCompartmentalModel()

## End(Not run)
```

save_grid *Save a plot grid*

Description

Writes the assembled ‘$grid‘ of a plot result to disk. No-op (with a message) when ‘$grid‘ is ‘NULL‘ (e.g. patchwork unavailable).

Usage

```
save_grid(res, path, limitsize = FALSE, ...)
```

Arguments

res	A plot result list containing a ‘$grid‘ element.
path	Output file path.
limitsize	Passed to [ggplot2::ggsave()].
...	Passed to [ggplot2::ggsave()].

Value

The output path, invisibly (or ‘NULL‘ if there is no grid).

Examples

```
## Not run:
res <- plot_fit(fit)            # fit from fitCompartmentalModel()
save_grid(res, file.path(tempdir(), "Plot.pdf"))

## End(Not run)
```

`setup_julia` *Initialise the Julia bridge*

Description

Starts the Julia session (via diffeqr + JuliaCall) and, on first use, provisions the required Julia packages (SciMLBase, OrdinaryDiffEq, Turing, Distributions) from a *pinned* environment shipped with the package (`inst/julia/Project.toml`), copied to a writable per-user cache and instantiated so every user gets the same tested versions. If that fails it falls back to installing the packages into the default Julia environment. Loads OrdinaryDiffEq. Safe to call repeatedly: the session bind and the one-time provisioning are each guarded.

Usage

```
setup_julia()
```

Value

TRUE, invisibly.

Examples

```
## Not run:
setup_julia()  # starts Julia; installs the Julia packages on first call

## End(Not run)
```

`simulate_model` *Simulate a fully-specified compartmental model (no fitting)*

Description

Solve a compartmental ODE model whose states and parameters are ALL fixed (‘*name=value‘), with no observed data and no estimation. The model is built, its fixed point recovered, solved over the time grid, and any ‘data_dummy‘ (and optional ‘dataCombined‘) formulas are evaluated on the trajectory.

Usage

```
simulate_model(
  modelParams,
  data_dummy = NULL,
  dataCombined = NULL,
  solver = solver_control()
)
```

Arguments

modelParams	Model parameter sheet (data frame). Every state and parameter must be fixed.
data_dummy	Optional dummy-data data frame (a 'Formula' column of expressions to evaluate on the trajectory; no likelihood, never fit).
dataCombined	Optional observed data to also evaluate as overlay series (never fit). 'NULL' (the default) simulates with dummy series only.
solver	Solver settings, see [solver_control()]. Use 'solver_control(backend = "r")' for a Julia-free solve.

Details

Use this to run a scenario forward from known values – e.g. a hand-specified SIR/SEIR model – without supplying 'dataCombined'. If any quantity is still fittable (a '[lo,hi]' box or a distributional prior), the sheet is not fully specified and an error is raised naming the offending quantities; fit those with [fitCompartmentalModel()] instead.

Value

An object of class '"compartmentalSim"': a list with 'initial_state', 'parms', 'sir_out' (solved trajectory), 'evaluation' (formulas over the grid), 'time_grid', 'model', 'solver', 'data_dummy' and 'dataCombined'.

See Also

[plot_simulation()], [build_compartmental_model()], [fitCompartmentalModel()].

Examples

```
sim_dir <- system.file("extdata", "SIR_sim", package = "compfit")
mp <- read_data_file(file.path(sim_dir, "modelParams.csv"))
dd <- read_data_file(file.path(sim_dir, "dataDummy.csv"))
sim <- simulate_model(mp, data_dummy = dd, solver = solver_control(backend = "r"))
head(sim$sir_out)
sim$evaluation
```

---

sobol_loss *Sobol sensitivity of the loss*

Description

Variance-based (Sobol) global sensitivity of the loss function to the fitted parameters, via [sensitivity::soboljansen()].

Usage

```
sobol_loss(
  x,
  n = 1000,
  dataCombined = NULL,
  solver = NULL,
  progress = TRUE,
  ...
)
```

Arguments

x	A ‘"compartmentalFit"‘ or ‘"compartmentalModel"‘ object.
n	Base sample size for the Sobol design.
dataCombined	Optional override data (defaults to the fit’s data).
solver	Optional solver settings (defaults to the fit’s solver).
progress	Show a progress bar.
...	Passed to [sensitivity::soboljansen()].

Value

A tidy data frame of first-order and total indices per parameter.

Examples

```
## Not run:
sobol_loss(fit, n = 500)   # fit or model from build_compartmental_model()

## End(Not run)
```

---

sobol_report *Summarise a Sobol sensitivity analysis*

Description

Reports validity, per-stream top parameters, interaction structure, and a global parameter ranking from a Sobol analysis. Accepts a precomputed tidy table or a fit/model (computed via [sobol_streams()]).

Usage

```
sobol_report(x, n = 500, cor_to = NULL, plots = TRUE, top_n = 5, ...)
```

Arguments

x	A tidy Sobol data frame (output, parameter, first_order, total) or a ‘"compartmentalFit"‘/‘"compartmentalModel"‘ object.
n	Base sample size when computing from a fit/model.
cor_to	Optional parameter name to correlate others against.
plots	If ‘TRUE‘, build a total-index heatmap.
top_n	Number of top parameters to report per stream.
...	Passed to [sobol_streams()] when computing.

Value

Invisibly, a list with the table, stream sums, parameter ranking, not-converged flags, range/NA diagnostics, and (if ‘plots‘) a heatmap.

Examples

```
## Not run:
sobol_report(fit, n = 500)            # compute from a fit, or:
sob <- sobol_streams(fit, n = 500)
r   <- sobol_report(sob)              # report a precomputed table

## End(Not run)
```

`sobol_streams` *Sobol sensitivity of model outputs*

Description

Variance-based (Sobol) global sensitivity of a summary of each model output stream to the fitted parameters.

Usage

```
sobol_streams(
  x,
  n = 500,
  summary_fun = mean,
  streams = NULL,
  dataCombined = NULL,
  solver = NULL,
  progress = TRUE,
  ...
)
```

Arguments

`x`	A ‘"compartmentalFit"‘ or ‘"compartmentalModel"‘ object.
`n`	Base sample size for the Sobol design.
`summary_fun`	Function summarising each output trajectory to a scalar.
`streams`	Optional subset of output streams to analyse.
`dataCombined`	Optional override data (defaults to the fit’s data).
`solver`	Optional solver settings (defaults to the fit’s solver).
`progress`	Show a progress bar.
`...`	Passed to [sensitivity::soboljansen()].

Value

A tidy data frame of first-order/total indices per output and parameter.

Examples

```
## Not run:
sob <- sobol_streams(fit, n = 500)

## End(Not run)
```

`solver_control` *ODE solver settings*

Description

Bundle the Julia ODE solver and tolerances used for fitting and plotting.

Usage

```
solver_control(
  solver = "AutoTsit5(Rosenbrock23())",
  abstol = 1e-08,
  reltol = 1e-08,
  backend = c("julia", "r", "stan")
)
```

Arguments

`solver`	ODE solver. Its meaning is backend-specific, and the default ‘"AutoTsit5(Rosenbrock23())"‘ maps to each backend’s auto-switching / non-stiff default, which suits almost every compartmental model. • **Julia**: the ‘OrdinaryDiffEq‘ constructor expression, passed verbatim. Common choices: ‘"Tsit5()"‘ (non-stiff), ‘"Vern7()"‘/‘"Vern9()"‘ (high-accuracy), ‘"Rosenbrock23()"‘/‘"Rodas5()"‘/‘"KenCarp4()"‘/‘"TRBDF2()"‘ (stiff), ‘"AutoVern7(Rodas5())"‘ (auto-switching). Any ‘OrdinaryDiffEq‘ solver works; see the DifferentialEquations.jl docs. • **R** (deSolve): a ‘deSolve::ode‘ ‘method‘ name, e.g. ‘"lsoda"‘ (default, auto-switching), ‘"radau"‘/‘"bdf"‘ (stiff), ‘"rk4"‘/‘"ode45"‘ (non-stiff). Unrecognised strings fall back to ‘"lsoda"‘. • **Stan**: ‘"rk45"‘ (default, non-stiff), ‘"bdf"‘ (stiff), or ‘"adams"‘ (non-stiff multistep). Set via ‘solver = "bdf"‘ etc.
`abstol`	Absolute tolerance (all backends).
`reltol`	Relative tolerance (all backends).
`backend`	Solver backend: ‘"julia"‘ (default; fast, needs a Julia session via [setup_julia()]), ‘"r"‘ (pure R via deSolve; no Julia required, but slower), or ‘"stan"‘ (Bayesian sampling via Stan/rstan NUTS – no Julia; all R-side solves, e.g. plotting and the optional MAP init, use deSolve).

Value

A named list of solver settings.

Examples

```
solver_control(abstol = 1e-6, reltol = 1e-6)
solver_control(backend = "r")   # Julia-free
```

solveWithJulia *Solve the ODE in Julia*

Description

Low-level solver bridge: solves the registered Julia ODE at a given initial state, parameter vector and save times. Mirrors [solveWithR()] (the deSolve backend) and returns the same shape. Exported because the scripts emitted by [extract_code()] call it.

Usage

```
solveWithJulia(
  X,
  t,
  p,
  time_integer,
  solver = "BS3()",
  abstol = 1e-08,
  reltol = 1e-08
)
```

Arguments

X	Named numeric initial-state vector.
t	Length-2 numeric integration span ‘c(t0, t1)‘.
p	Numeric parameter vector (in ODE order).
time_integer	Numeric vector of save times.
solver	Julia solver expression.
abstol, reltol	Solver tolerances.

Value

A list with ‘matrix‘ (states by time) and ‘t‘ (the save times).

solve_and_evaluate *Solve the ODE and evaluate formulas*

Description

Solves the model at a given ‘(initial_state, parms)‘ and evaluates every data (and optional dummy) formula on the resulting trajectory. Self-sufficient: reads the time grid, solver settings and formulas from the ‘fit‘ object.

Usage

```
solve_and_evaluate(fit, initial_state, parms, data_dummy = NULL)
```

Arguments

fit	A ‘"compartmentalFit"‘ object.
initial_state	Named numeric vector of initial compartment values.
parms	Named numeric vector of parameters.
data_dummy	Optional dummy-data data frame; ‘NULL‘ evaluates only the data streams.

Value

A list with ‘sir_out‘ (trajectory) and ‘evaluation‘ (formula columns).

Examples

```
## Not run:
# fit from fitCompartmentalModel(); evaluate at the fitted point
p  <- get_point(fit)
ev <- solve_and_evaluate(fit, p$initial_state, p$parms)
head(ev$evaluation)

## End(Not run)
```

---

summary.compartmentalFit

Summary of a compartmental fit

Description

Summary of a compartmental fit

Usage

```
## S3 method for class 'compartmentalFit'
summary(object, ...)
```

Arguments

object	A ‘"compartmentalFit"‘ object.
...	Unused.

Value

‘object‘, invisibly (called for the printed summary).

Examples

```
## Not run:
summary(fit)   # fit from fitCompartmentalModel()

## End(Not run)
```

validate_modelParams *Validate a modelParams sheet*

Description

Check a model-parameter sheet for common entry errors before it is built. On any problem it raises a single error listing every issue found, each naming the column/cell and what is wrong. Checks:

- fixed `States`/`Parameters` values and `Linear`/`Quadratic`/`Constant`/`Functions`/`Conditions` expressions reference only declared symbols (parameters, `_0` aliases, the declared states – `X1..Xn` or named, e.g. `S/I/R` –, functions, `time`), so a typo like `*2*-bta` is caught;
- box priors `[lo,hi]` are two numbers with `lo < hi`;
- distribution priors have numeric arguments;
- no parameter/state/function name collides with a Julia keyword or a codegen variable/function (`t`/`p`/`X`/`du`/`dX`/ `parms`, `N1..`, `total_pop`, `f<ij>`, `g<ij>`, `cst<i>`, `secOrd_i_j`, or another quantity's `_0` alias);
- `Quadratic` cells are `*goto*coeff` with a target in `1..n`;
- `Others` has numeric `startpoint`/`endpoint`/`partition`.

Usage

```
validate_modelParams(modelParams)
```

Arguments

`modelParams`	The model-parameter data frame (as read by `load_scenario()`).

Value

Invisibly `TRUE` when valid; otherwise stops with the collected problems.

Examples

```
mini <- system.file("extdata", "minimal", package = "compfit")
sc <- load_scenario(mini, combined_file = "dataCombined.csv",
                    dummy_file = "dataDummy.csv", params_file = "modelParams.csv")
validate_modelParams(sc$modelParams)   # TRUE (the fixture is valid)
```

verify_filled *Verify a filled model sheet*

Description

Re-reads a filled workbook and checks it rebuilds the same model structure as the original sheet (catches accidental structural edits).

Usage

```
verify_filled(filled_path, modelParams)
```

Arguments

filled_path	Path to the filled ‘.xlsx‘.
modelParams	The original model parameter sheet (data frame).

Value

‘TRUE‘, invisibly, if all checks pass; otherwise an error.

Examples

```
## Not run:
verify_filled(file.path(tempdir(), "modelParams_filled.xlsx"), sc$modelParams)

## End(Not run)
```

---

write_filled_params *Write a filled model sheet to a workbook*

Description

Fills the sheet via [fill_params()] and writes it to an ‘.xlsx‘ in the scenario directory.

Usage

```
write_filled_params(
  fit,
  scenario_dir,
  modelParams = fit$model$modelParams,
  out_file = "modelParams_filled.xlsx",
  digits = NULL,
  summary = c("median", "mean")
)
```

Arguments

fit	A ‘"compartmentalFit"‘ object.
scenario_dir	Directory to write the workbook into.
modelParams	The model parameter sheet (defaults to the fit’s stored sheet).
out_file	Output workbook name.
digits	Optional rounding for written values.
summary	Which point estimate to write (‘"median"‘ or ‘"mean"‘).

Value

The output path, invisibly.

Examples

```
## Not run:
write_filled_params(fit, tempdir(), modelParams = sc$modelParams)

## End(Not run)
```

Index